

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

LAB TASK 3

BECE406E

FPGA System Design Lab

NAME	KAARMUKILAN A G
REGISTRATION NO	23BEC0137
SLOT	L33+L34
FACULTY NAME	AARTHY M

AIM:

i) Create a text file which contains Mod 50 counter values. Read the content of the text file and store in single port RAM. Create a blank text file and write the Mod 50 counter values. Compare both files and display whether values are matched or not. Attach screenshots of code, simulation, both text files.

ii) Generate a dual port RAM IP using Quartus and verify the functionality using testbench in Modelsim. Attach the RTL view of RAM and Screenshots of code and outputs.

SOFTWARE DETAILS:

Functional Simulation: Modelsim
Synthesis & Implementation: Intel Quartus Prime 23.1

FILE-HANDLING

RTL CODE and TESTBENCH

```
module file(
    input clk,
    input en,
    input [5:0] addr,
    output reg [7:0] dataout
);

reg [7:0] mem [0:49];

always @(posedge clk) begin
    if (en)
        dataout <= mem[addr];
end

endmodule

module mod50(
    input clk,
    input rst,
    output reg [7:0] count
);

always @(posedge clk or posedge rst) begin
    if (rst)
        count <= 0;
    else if (count == 8'd49)
        count <= 0;
    else
        count <= count + 1;
end

endmodule
```

```
module tb_file;
reg clk, rst;
reg en;
reg [5:0] addr;
wire [7:0] count;
wire [7:0] dataout;
integer f1, f2, i;
integer flag;
reg [7:0] mem1 [0:49];
reg [7:0] mem2 [0:49];
mod50 C1(clk, rst, count);
file DUT(clk, en, addr, dataout);
always #5 clk = ~clk;
initial begin
    clk = 0;    rst = 1;    en  = 0;    flag = 1;
    f1 = $fopen("file137_1.txt", "w");
    #10 rst = 0;
    for(i = 0; i < 50; i = i + 1) begin
       @(posedge clk);
        $fdisplay(f1, "%d", count);
    end
    $fclose(f1);//writing mod50 into file1
    $readmemh("file137_1.txt", DUT.mem);
    f2 = $fopen("file137_2.txt", "w");
    for(i = 0; i < 50; i = i + 1)
        $fdisplay(f2, "%h", DUT.mem[i]);
    $fclose(f2);//comparing both files
    $readmemh("file137_1.txt", mem1);
    $readmemh("file137_2.txt", mem2);
    for(i = 0; i < 50; i = i + 1)
        if(mem1[i] != mem2[i])
            flag = 0;
    if(flag)        $display("Files are MATCHED");
    else        $display("Files are NOT MATCHED");
    $finish;
end
endmodule
```

```
module file(
   input clk,
   input en,
   input [5:0] addr,
   output reg [7:0] dataout
);

reg [7:0] mem [0:49];

always @(posedge clk) begin
   if (en)
      dataout <= mem[addr];
end

endmodule

module mod50(
   input clk,
   input rst,
   output reg [7:0] count
);

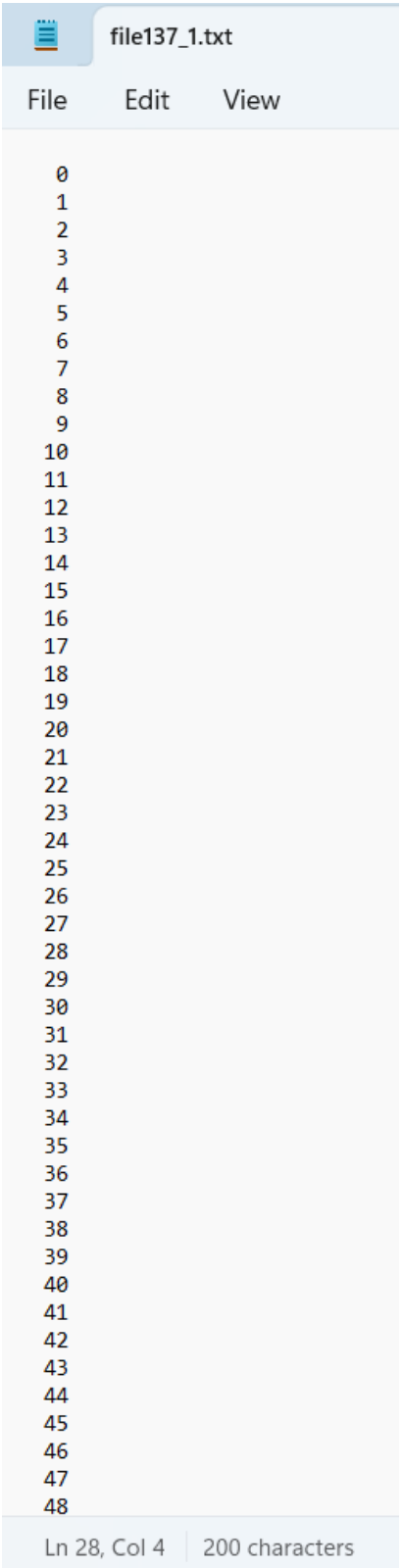
always @(posedge clk or posedge rst) begin
   if (rst)
      count <= 0;
   else if (count == 8'd49)
      count <= 0;
   else
      count <= count + 1;
end

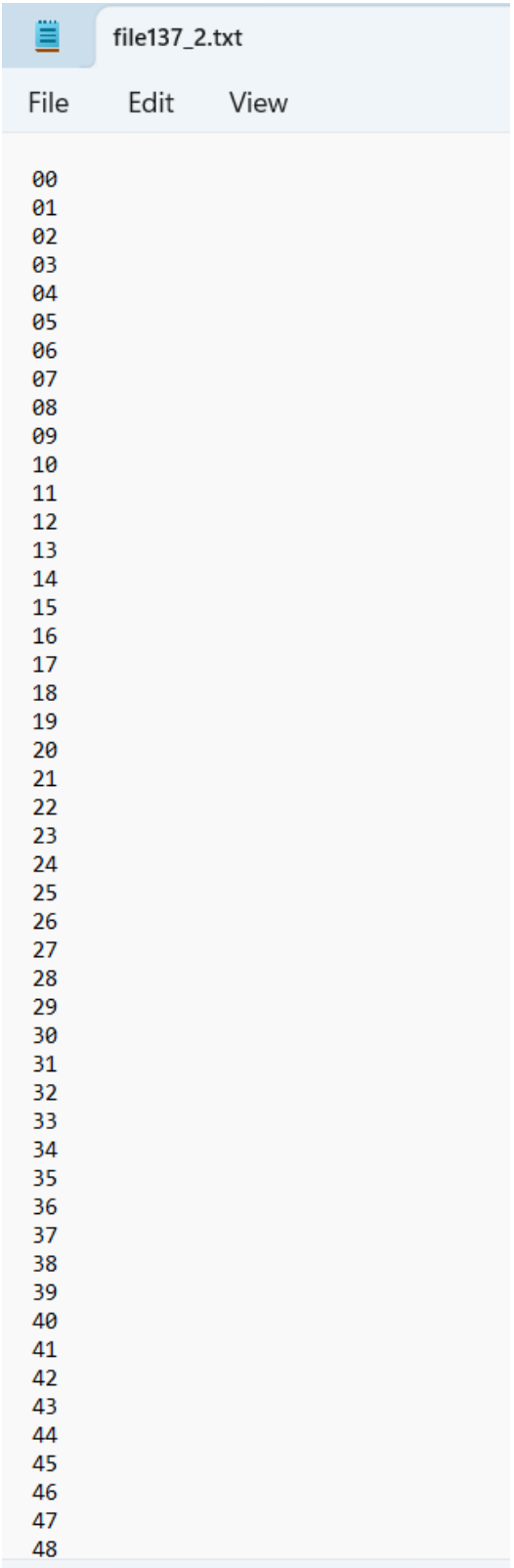
endmodule
```

```
module tb_file;
reg clk, rst;
reg en;
reg [5:0] addr;
wire [7:0] count;
wire [7:0] dataout;
integer f1, f2, i;
integer flag;
reg [7:0] mem1 [0:49];
reg [7:0] mem2 [0:49];
mod50 C1(clk, rst, count);
file DUT(clk, en, addr, dataout);
always #5 clk = ~clk;
initial begin
   clk = 0;    rst = 1;    en  = 0;    flag = 1;
   f1 = $fopen("file137_1.txt", "w");
   #10 rst = 0;
   for(i = 0; i < 50; i = i + 1) begin
     @(posedge clk);
      $fdisplay(f1, "%d", count);
   end
   $fclose(f1);//writing mod50 into file1
   $readmemh("file137_1.txt", DUT.mem);
   f2 = $fopen("file137_2.txt", "w");
   for(i = 0; i < 50; i = i + 1)
      $fdisplay(f2, "%h", DUT.mem[i]);
   $fclose(f2);//comparing both files
   $readmemh("file137_1.txt", mem1);
   $readmemh("file137_2.txt", mem2);
   for(i = 0; i < 50; i = i + 1)
      if(mem1[i] != mem2[i])
```

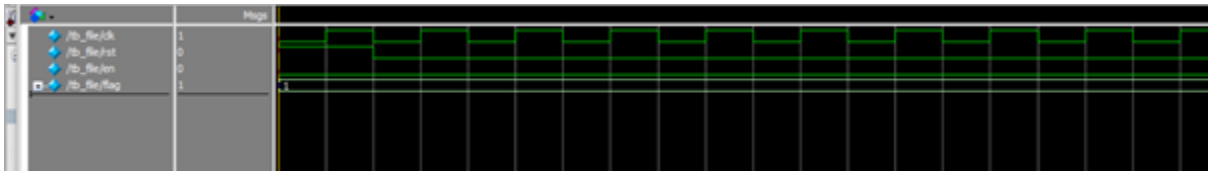
```
        flag = 0;
    if(flag)        $display("Files are MATCHED");
    else        $display("Files are NOT MATCHED");
        $finish;
    end
    endmodule
```

SCREENSHOTS OF TEXT FILES



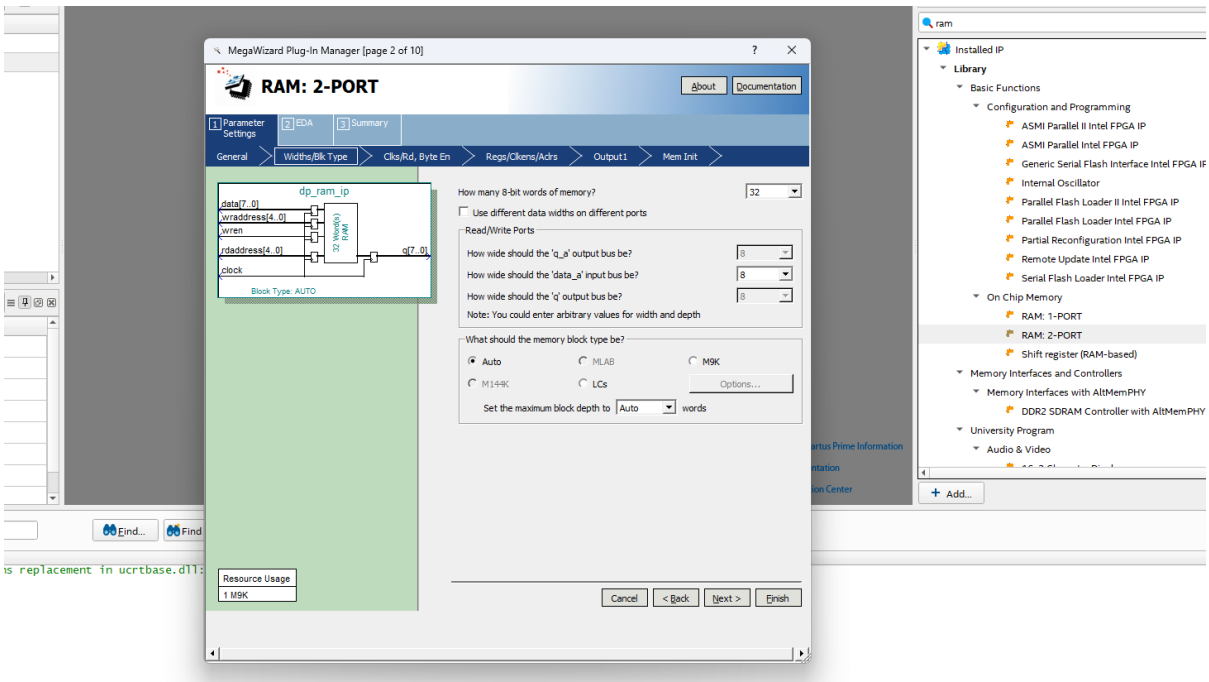


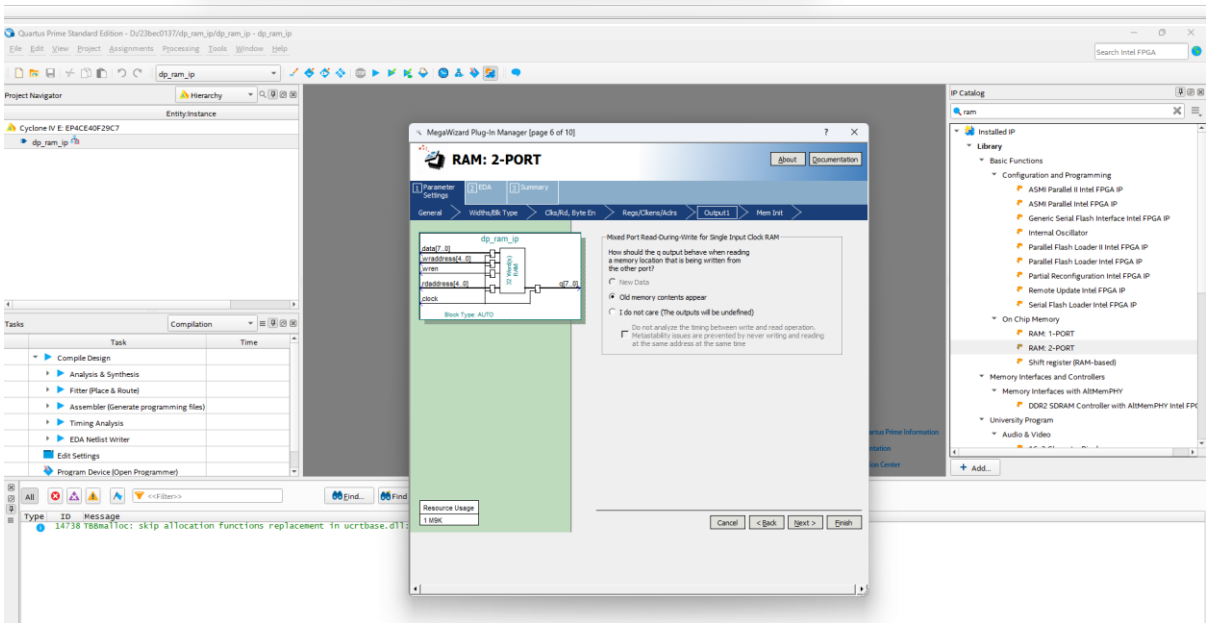
FUNCTIONAL SIMULATION:

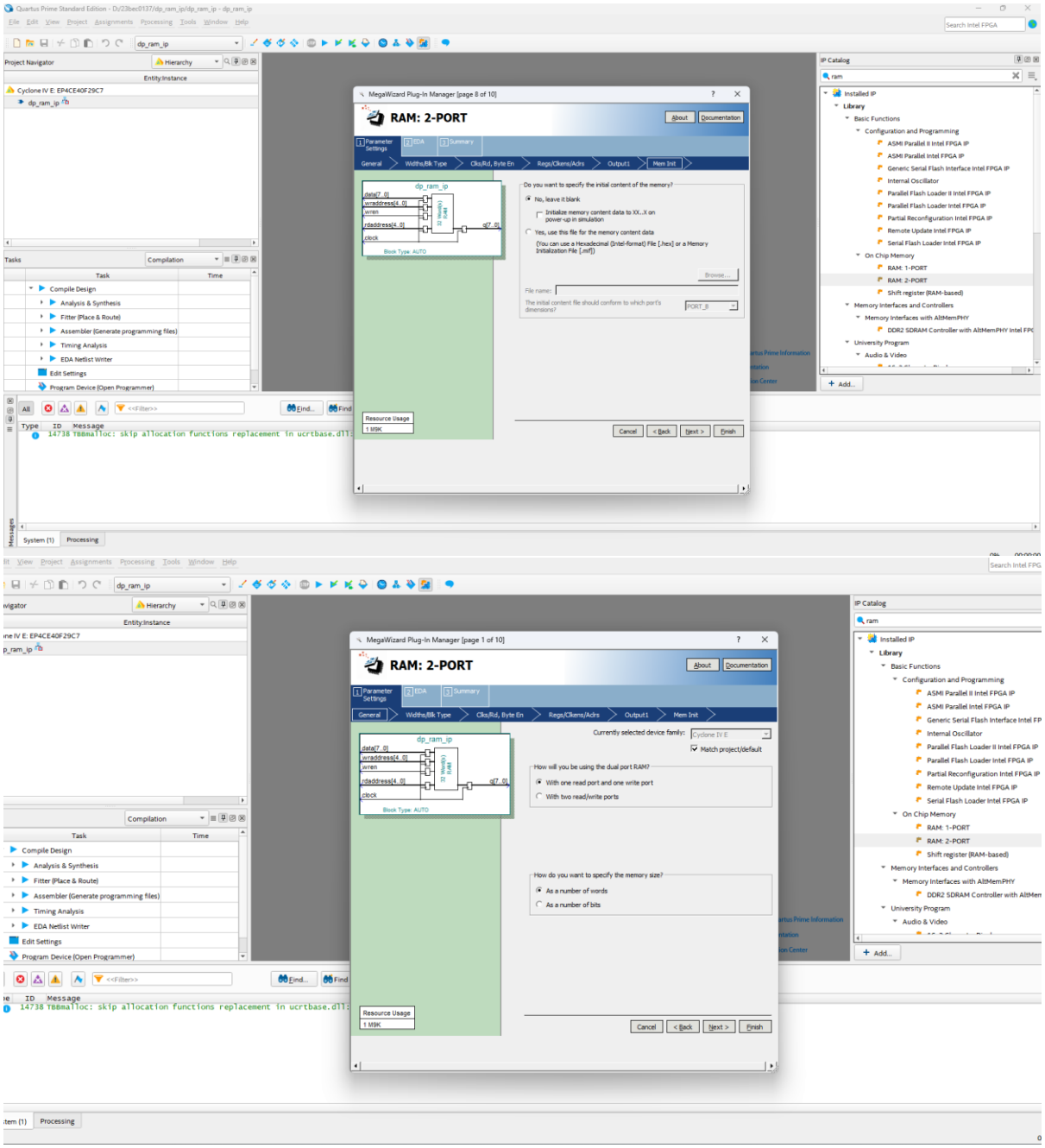


DUAL PORT RAM IP

Steps for IP Creation in IP Catalog In Quartus Prime






MegaWizard Plug-In Manager [page 8 of 10]
RAM: 2-PORT
About
Documentation
Parameter Settings
EDA
Summary
General
Widths/Blk Type
Clks/Rd, Byte En
Regs/Clkens/Adrs
Output1
Mem Init
dp_ram_ip
data[7..0]
wraddress[4..0]
wren
rdaddress[4..0]
clock
q[7..0]
Block Type: AUTO
Do you want to specify the initial content of the memory?
No, leave it blank
Initialize memory content data to XX...X on power-up in simulation
Yes, use this file for the memory content data
(You can use a Hexadecimal (Intel-format) File [.hex] or a Memory Initialization File [.mif])
Browse...
File name:
The initial content file should conform to which port's dimensions?
PORT_B
Resource Usage
1 M9K
Cancel
< Back
Next >
Finish
IP Catalog
ram
Installed IP
Library
Basic Functions
Configuration and Programming
ASMI Parallel II Intel FPGA IP
ASMI Parallel Intel FPGA IP
Generic Serial Flash Interface Intel FPGA IP
Internal Oscillator
Parallel Flash Loader II Intel FPGA IP
Parallel Flash Loader Intel FPGA IP
Partial Reconfiguration Intel FPGA IP
Remote Update Intel FPGA IP
Serial Flash Loader Intel FPGA IP
On Chip Memory
RAM: 1-PORT
RAM: 2-PORT
Shift register (RAM-based)
Memory Interfaces and Controllers
Memory Interfaces with AltMemPHY
DDR2 SDRAM Controller with AltMemPHY Intel FPGA IP
University Program
Audio & Video
Add...
Tasks
Compilation
Task
Time
Compile Design
Analysis & Synthesis
Fitter (Place & Route)
Assembler (Generate programming files)
Timing Analysis
EDA Netlist Writer
Edit Settings
Program Device (Open Programmer)
Type
ID
Message
14738 TBBmalloc: skip allocation functions replacement in ucrtbase.dll
System (1)
Processing
MegaWizard Plug-In Manager [page 1 of 10]
How will you be using the dual port RAM?
With one read port and one write port
With two read/write ports
How do you want to specify the memory size?
As a number of words
As a number of bits
Currently selected device family:
Cyclone IV E
Match project/default

RTL CODE AND TESTBENCH:

```
// synopsys translate_off
`timescale 1 ps / 1 ps
// synopsys translate_on
module dp_ram_ip (
        clock,
        data,
        rdaddress,
        wraddress,
        wren,
        q);

        input     clock;
        input   [7:0]  data;
        input   [4:0]  rdaddress;
        input   [4:0]  wraddress;
        input     wren;
        output  [7:0]  q;
`ifndef ALTERA_RESERVED_QIS
// synopsys translate_off
`endif
        tri1      clock;
        tri0      wren;
`ifndef ALTERA_RESERVED_QIS
// synopsys translate_on
`endif

        wire [7:0] sub_wire0;
        wire [7:0] q = sub_wire0[7:0];

        altsyncram      altsyncram_component (
                                .address_a (wraddress),
                                .address_b (rdaddress),
                                .clock0 (clock),
                                .data_a (data),
                                .wren_a (wren),
                                .q_b (sub_wire0),
                                .aclr0 (1'b0),
                                .aclr1 (1'b0),
                                .addressstall_a (1'b0),
                                .addressstall_b (1'b0),
                                .byteena_a (1'b1),
                                .byteena_b (1'b1),
                                .clock1 (1'b1),
                                .clocken0 (1'b1),
```

```
                                .aclr0 (1'b0),
                                .aclr1 (1'b0),
                                .addressstall_a (1'b0),
                                .addressstall_b (1'b0),
                                .byteena_a (1'b1),
                                .byteena_b (1'b1),
                                .clock1 (1'b1),
                                .clocken0 (1'b1),
                                .clocken1 (1'b1),
                                .clocken2 (1'b1),
                                .clocken3 (1'b1),
                                .data_b ({8{1'b1}}),
                                .eccstatus (),
                                .q_a (),
                                .rden_a (1'b1),
                                .rden_b (1'b1),
                                .wren_b (1'b0));
        defparam
                altsyncram_component.address_aclr_b = "NONE",
                altsyncram_component.address_reg_b = "CLOCK0",
                altsyncram_component.clock_enable_input_a = "BYPASS",
                altsyncram_component.clock_enable_input_b = "BYPASS",
                altsyncram_component.clock_enable_output_b = "BYPASS",
                altsyncram_component.intended_device_family = "Cyclone IV E",
                altsyncram_component.lpm_type = "altsyncram",
                altsyncram_component.numwords_a = 32,
                altsyncram_component.numwords_b = 32,
                altsyncram_component.operation_mode = "DUAL_PORT",
                altsyncram_component.outdata_aclr_b = "NONE",
                altsyncram_component.outdata_reg_b = "CLOCK0",
                altsyncram_component.power_up_uninitialized = "FALSE",
                altsyncram_component.read_during_write_mode_mixed_ports = "OLD_DATA",
                altsyncram_component.widthad_a = 5,
                altsyncram_component.widthad_b = 5,
                altsyncram_component.width_a = 8,
                altsyncram_component.width_b = 8,
                altsyncram_component.width_byteena_a = 1;


endmodule
```

```
`timescale 1ns/1ps

module tb_dpram;
reg clk;
reg [7:0] din;
reg [4:0] raddr;
reg [4:0] waddr;
reg we;
wire [7:0] q;

dp_ram_ip U1 (
    .clock(clk),
    .data(din),
    .rdaddress(raddr),
    .wraddress(waddr),
    .wren(we),
    .q(q)
);
always #5 clk = ~clk;
initial begin
    clk = 0;
    din = 0;
    raddr = 0;
    waddr = 0;
    we = 0;
    // WRITE
    #10 we = 1; waddr = 5'd1;  din = 8'h11;
    #10 we = 1; waddr = 5'd5;  din = 8'h55;
    #10 we = 1; waddr = 5'd10; din = 8'hAA;
    // READ
    #10 we = 0; raddr = 5'd1;
    #10 raddr = 5'd5;
    #10 raddr = 5'd10;
    // SIMULTANEOUS READ + WRITE
    #10 we = 1; waddr = 5'd15; din = 8'hF0;
        raddr = 5'd5;
    // SAME ADDRESS
    #10 waddr = 5'd20; raddr = 5'd20; din = 8'h77;
    #10 we = 0;
    #20 $finish;
end
endmodule
```

```
`timescale 1 ps / 1 ps
// synopsys translate_on
module dp_ram_ip (
        clock,
        data,
        rdaddress,
        wraddress,
        wren,
        q);

        input     clock;
        input   [7:0]  data;
        input   [4:0]  rdaddress;
        input   [4:0]  wraddress;
        input     wren;
        output  [7:0]  q;
`ifndef ALTERA_RESERVED_QIS
// synopsys translate_off
`endif
        tri1      clock;
        tri0      wren;
`ifndef ALTERA_RESERVED_QIS
// synopsys translate_on
`endif

        wire [7:0] sub_wire0;
        wire [7:0] q = sub_wire0[7:0];

        altsyncram   altsyncram_component (
                                .address_a
(wraddress),
                                .address_b
(rdaddress),
                                .clock0 (clock),
                                .data_a (data),
                                .wren_a (wren),
                                .q_b (sub_wire0),
                                .aclr0 (1'b0),
```

```
                                .aclr1 (1'b0),
                                .addressstall_a (1'b0),
                                .addressstall_b (1'b0),
                                .byteena_a (1'b1),
                                .byteena_b (1'b1),
                                .clock1 (1'b1),
                                .clocken0 (1'b1),
                                .clocken1 (1'b1),
                                .clocken2 (1'b1),
                                .clocken3 (1'b1),
                                .data_b ({8{1'b1}}),
                                .eccstatus (),
                                .q_a (),
                                .rden_a (1'b1),
                                .rden_b (1'b1),
                                .wren_b (1'b0));
        defparam
                altsyncram_component.address_aclr_b =
"NONE",
                altsyncram_component.address_reg_b =
"CLOCK0",
                altsyncram_component.clock_enable_input_a =
"BYPASS",
                altsyncram_component.clock_enable_input_b =
"BYPASS",
                altsyncram_component.clock_enable_output_b
= "BYPASS",
                altsyncram_component.intended_device_family
= "Cyclone IV E",
                altsyncram_component.lpm_type =
"altsyncram",
                altsyncram_component.numwords_a = 32,
                altsyncram_component.numwords_b = 32,
                altsyncram_component.operation_mode =
"DUAL_PORT",
                altsyncram_component.outdata_aclr_b =
"NONE",
                altsyncram_component.outdata_reg_b =
"CLOCK0",
                altsyncram_component.power_up_uninitialized
= "FALSE",
                altsyncram_component.read_during_write_mode_mixed_por
ts = "OLD_DATA",
                altsyncram_component.widthad_a = 5,
                altsyncram_component.widthad_b = 5,
                altsyncram_component.width_a = 8,
                altsyncram_component.width_b = 8,
                altsyncram_component.width_byteena_a = 1;


endmodule
```

```
`timescale 1ns/1ps

module tb_dpram;

reg clk;
reg [7:0] din;
reg [4:0] raddr;
reg [4:0] waddr;
reg we;

wire [7:0] q;

dp_ram_ip U1 (
    .clock(clk),
    .data(din),
    .rdaddress(raddr),
    .wraddress(waddr),
    .wren(we),
    .q(q)
);

always #5 clk = ~clk;

initial begin

    clk = 0;
    din = 0;
    raddr = 0;
    waddr = 0;
    we = 0;

    // WRITE
    #10 we = 1; waddr = 5'd1;  din = 8'h11;
    #10 we = 1; waddr = 5'd5;  din = 8'h55;
    #10 we = 1; waddr = 5'd10; din = 8'hAA;

    // READ
    #10 we = 0; raddr = 5'd1;
    #10 raddr = 5'd5;
    #10 raddr = 5'd10;

    // SIMULTANEOUS READ + WRITE
    #10 we = 1; waddr = 5'd15; din = 8'hF0;
        raddr = 5'd5;

    // SAME ADDRESS
    #10 waddr = 5'd20; raddr = 5'd20; din = 8'h77;

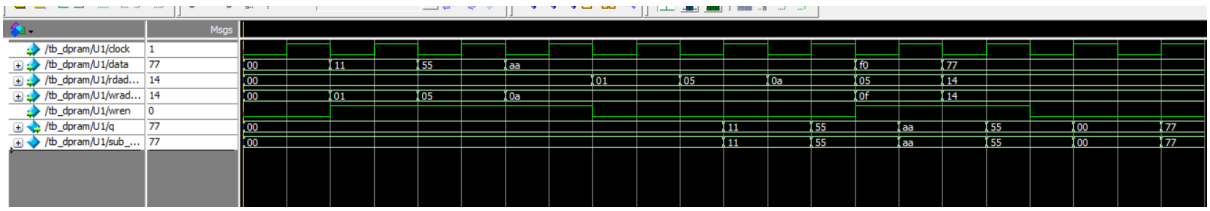
    #10 we = 0;

    #20 $finish;

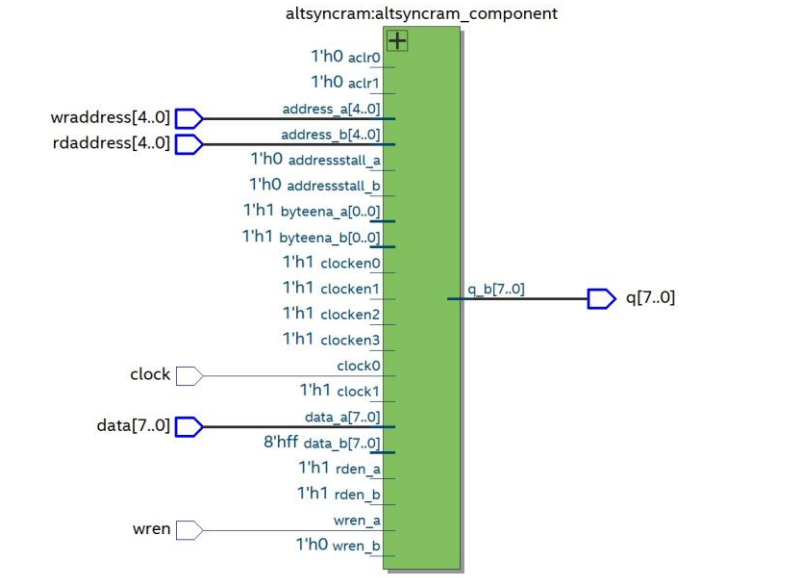
end


endmodule
```

FUNCTIONAL SIMULATION:



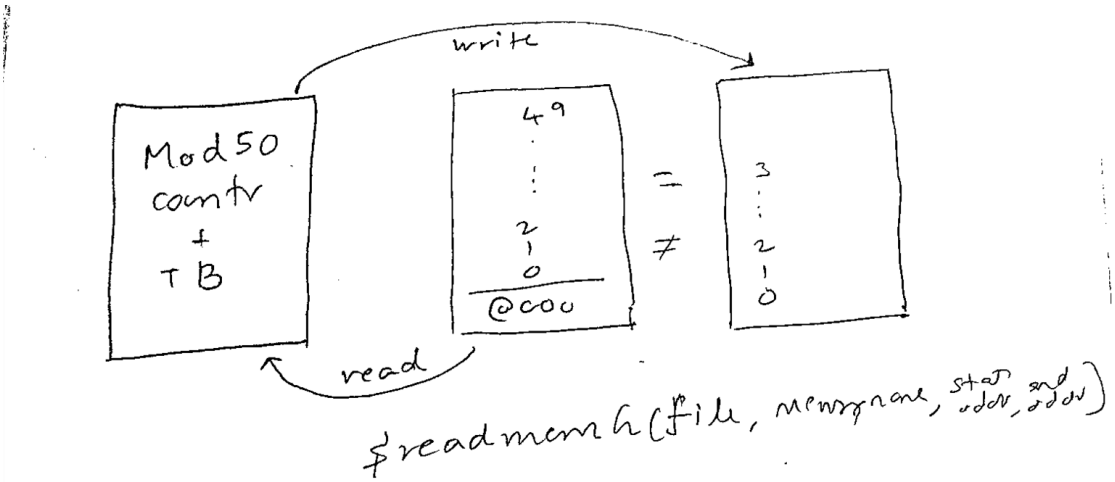
RTL SCHEMATIC:



INFERENCE:

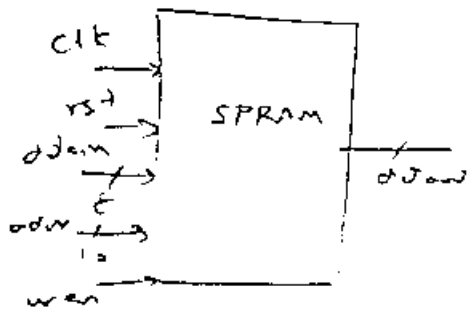
Hence, dual port ip has been generated and verified,

ATTESTATION :



FILE HANDLING
23BEC0137
KAARMUKILAN A.G

R/W both

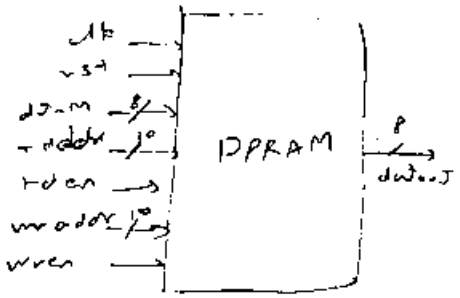


1024 X 8

2^{10}

SINGLE PORT RAM
23BEC0137
KAARMUKILAN.A.G

Minimum 2 clk cycles



DUAL PORT RAM
23BEC0137
KAARMUKILAN.A.G

Minimum 2 clk cycles